

CONTROL AND OBSERVABILITY

Track cost and usage

Learn how to track token usage, estimate costs, and configure prompt caching with the Claude Agent SDK.

The Claude Agent SDK provides detailed token usage information for each interaction with Claude. This guide explains how to properly track usage and understand cost reporting, especially when dealing with parallel tool uses and multi-step conversations.

For complete API documentation, see the [TypeScript SDK reference](#) and [Python SDK reference](#).

⚠ The `total_cost_usd` and `costUSD` fields are client-side estimates, not authoritative billing data. The SDK computes them locally from a price table bundled at build time, so they can drift from what you are actually billed when:

- pricing changes
- the installed SDK version does not recognize a model
- billing rules apply that the client cannot model

Use these fields for development insight and approximate budgeting. For authoritative billing, use the [Usage and Cost API](#) or the Usage page in the [Claude Console](#). Do not bill end users or trigger financial decisions from these fields.

Understand token usage

The TypeScript and Python SDKs expose the same usage data with different field names:

- **TypeScript** provides per-step token breakdowns on each assistant message (`message.message.id` , `message.message.usage`), per-model cost via `modelUsage` on the result message, and a cumulative total on the result message.
- **Python** provides per-step token breakdowns on each assistant message (`message.usage` , `message.message_id`), per-model cost via `model_usage` on the result message, and the accumulated total on the result message (`total_cost_usd` and `usage` dict).

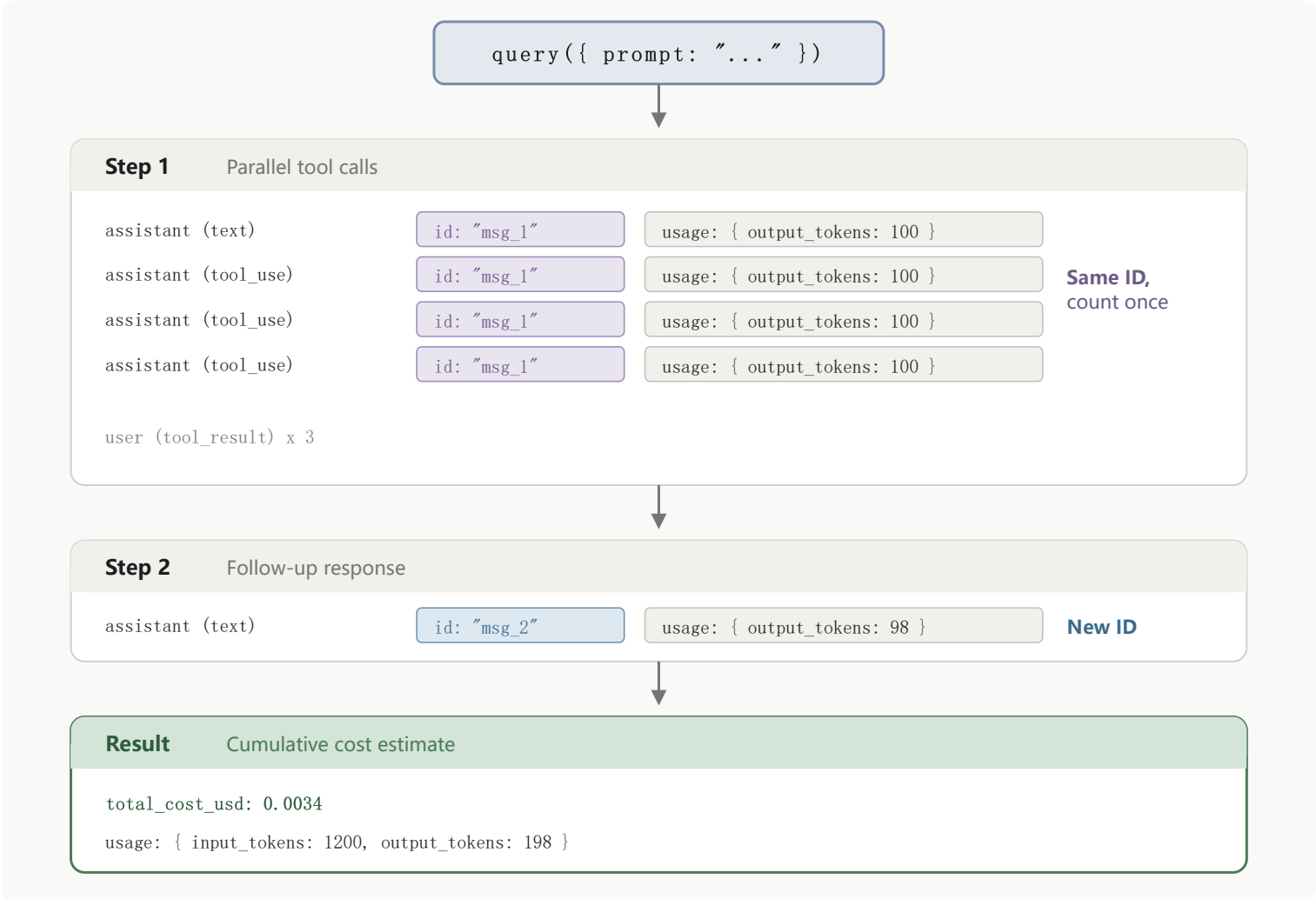
Both SDKs use the same underlying cost model and expose the same granularity. The difference is in

field naming and where per-step usage is nested.

Cost tracking depends on understanding how the SDK scopes usage data:

- query()** **call**: one invocation of the SDK's `query()` function. A single call can involve multiple steps (Claude responds, uses tools, gets results, responds again). Each call produces one **result** message at the end.
- Step**: a single request/response cycle within a `query()` call. Each step produces assistant messages with token usage.
- Session**: a series of `query()` calls linked by a session ID (using the `resume` option). Each `query()` call within a session reports its own cost independently.

The following diagram shows the message stream from a single `query()` call, with token usage reported at each step and the cumulative estimate at the end:



1 Each step produces assistant messages

When Claude responds, it sends one or more assistant messages. In TypeScript, each assistant message contains a nested `BetaMessage` (accessed via `message.message`) with an `id` and

a `usage` object with token counts (`input_tokens` , `output_tokens`). In Python, the `AssistantMessage` dataclass exposes the same data directly via `message.usage` and `message.message_id` . When Claude uses multiple tools in one turn, all messages in that turn share the same ID, so deduplicate by ID to avoid double-counting.

2 The result message provides the cumulative estimate

When the `query()` call completes, the SDK emits a result message with `total_cost_usd` and cumulative `usage` . This is available in both TypeScript (`SDKResultMessage`) and Python (`ResultMessage`). If you make multiple `query()` calls (for example, in a multi-turn session), each result only reflects the cost of that individual call. If you only need the estimated total, you can ignore the per-step usage and read this single value.

Get the total cost of a query

The result message ([TypeScript](#), [Python](#)) marks the end of the agent loop for a `query()` call. It includes `total_cost_usd` , the cumulative estimated cost across all steps in that call. This works for both success and error results. If you use sessions to make multiple `query()` calls, each result only reflects the cost of that individual call.

The following examples iterate over the message stream from a `query()` call and print the total cost when the `result` message arrives:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({ prompt: "Summarize this project" })) {
  if (message.type === "result") {
    console.log(`Total cost: ${message.total_cost_usd}`);
  }
}
```

Track per-step and per-model usage

The examples in this section use TypeScript field names. In Python, the equivalent fields are `AssistantMessage.usage` and `AssistantMessage.message_id` for per-step usage, and `ResultMessage.model_usage` for per-model breakdowns.

Track per-step usage

Each assistant message contains a nested `BetaMessage` (accessed via `message.message`) with an `id` and `usage` object with token counts. When Claude uses tools in parallel, multiple messages share the same `id` with identical usage data. Track which IDs you've already counted and skip duplicates to avoid inflated totals.

⚠ Parallel tool calls produce multiple assistant messages whose nested `BetaMessage` shares the same `id` and identical usage. Always deduplicate by ID to get accurate per-step token counts.

The following example accumulates input and output tokens across all steps, counting each unique message ID only once:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

const seenIds = new Set<string>();
let totalInputTokens = 0;
let totalOutputTokens = 0;

for await (const message of query({ prompt: "Summarize this
project" }))) {
  if (message.type === "assistant") {
    const msgId = message.message.id;

    // Parallel tool calls share the same ID, only count once
    if (!seenIds.has(msgId)) {
      seenIds.add(msgId);
      totalInputTokens += message.message.usage.input_tokens;
      totalOutputTokens += message.message.usage.output_tokens;
    }
  }
}

console.log(`Steps: ${seenIds.size}`);
console.log(`Input tokens: ${totalInputTokens}`);
console.log(`Output tokens: ${totalOutputTokens}`);
```

Break down usage per model

The result message includes `modelUsage`, a map of model name to per-model token counts and cost. This is useful when you run multiple models (for example, Haiku for subagents and Opus for the

main agent) and want to see where tokens are going.

The following example runs a query and prints the cost and token breakdown for each model used:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({ prompt: "Summarize this
project" })) {
  if (message.type !== "result") continue;

  for (const [modelName, usage] of
Object.entries(message.modelUsage)) {
    console.log(`${modelName}: ${usage.costUSD.toFixed(4)}`);
    console.log(`  Input tokens: ${usage.inputTokens}`);
    console.log(`  Output tokens: ${usage.outputTokens}`);
    console.log(`  Cache read: ${usage.cacheReadInputTokens}`);
    console.log(`  Cache creation:
${usage.cacheCreationInputTokens}`);
  }
}
```

Accumulate costs across multiple calls

Each `query()` call returns its own `total_cost_usd`. The SDK does not provide a session-level total, so if your application makes multiple `query()` calls (for example, in a multi-turn session or across different users), accumulate the totals yourself.

The following examples run two `query()` calls sequentially, add each call's `total_cost_usd` to a running total, and print both the per-call and combined cost:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Track cumulative cost across multiple query() calls
let totalSpend = 0;

const prompts = [
  "Read the files in src/ and summarize the architecture",
  "List all exported functions in src/auth.ts"
];

for (const prompt of prompts) {
  for await (const message of query({ prompt })) {
    if (message.type === "result") {
      totalSpend += message.total_cost_usd;
      console.log(`This call: ${message.total_cost_usd}`);
    }
  }
}

console.log(`Total spend: ${totalSpend.toFixed(4)}`);
```

Handle errors, caching, and token discrepancies

For accurate cost tracking, account for failed conversations, cache token pricing, and occasional reporting inconsistencies.

Resolve output token discrepancies

In rare cases, you might observe different `output_tokens` values for messages with the same ID. When this occurs:

1. **Use the highest value:** the final message in a group typically contains the accurate total.
2. **Prefer the result message:** the `total_cost_usd` in the result message reflects the SDK's accumulated estimate across all steps, so it is more reliable than summing per-step values yourself. It is still an estimate and may differ from your actual bill.
3. **Report inconsistencies:** file issues at the [Claude Code GitHub repository](#).

Track costs on failed conversations

Both success and error result messages include `usage` and `total_cost_usd` . If a conversation fails mid-way, you still consumed tokens up to the point of failure. Always read cost data from the result message regardless of its `subtype` .

Track cache tokens

The Agent SDK automatically uses **prompt caching** to reduce costs on repeated content. You do not need to configure caching yourself. The usage object includes two additional fields for cache tracking:

- `cache_creation_input_tokens` : tokens used to create new cache entries (charged at a higher rate than standard input tokens).
- `cache_read_input_tokens` : tokens read from existing cache entries (charged at a reduced rate).

Track these separately from `input_tokens` to understand caching savings. In TypeScript, these fields are typed on the `Usage` object. In Python, they appear as keys in the `ResultMessage.usage` dict (for example, `message.usage.get("cache_read_input_tokens", 0)`).

Extend the prompt cache TTL to one hour

Cache entries written by the SDK use a 5-minute TTL by default when you authenticate with an API key or run on Amazon Bedrock, Google Cloud Vertex AI, or Microsoft Foundry. If your workload runs many short sessions against the same system prompt and context with gaps longer than 5 minutes between them, the cache expires between sessions and each new session pays full input price.

To request a 1-hour TTL on cache writes, set the `ENABLE_PROMPT_CACHING_1H` environment variable. You can export it in your shell or container environment, or pass it through `options.env` .

The following example enables 1-hour TTL for an agent running on Bedrock:

```

from claude_agent_sdk import ClaudeAgentOptions, query
import asyncio

async def main():
    options = ClaudeAgentOptions(
        env={
            "CLAUDE_CODE_USE_BEDROCK": "1",
            "ENABLE_PROMPT_CACHING_1H": "1",
        },
    )

    async for message in query(prompt="Summarize this project",
options=options):
        print(message)

asyncio.run(main())

```

Cache writes with a 1-hour TTL are billed at a higher rate than 5-minute writes, so enabling this trades higher write cost for more cache reads. See [prompt caching pricing](#) for details. Claude subscription users already receive 1-hour TTL automatically and do not need to set this variable.

Related documentation

- [TypeScript SDK Reference](#) - Complete API documentation
- [SDK Overview](#) - Getting started with the SDK
- [SDK Permissions](#) - Managing tool permissions